

Software Design Document

Evidence-driven Method-level Vulnerability Risk Assessment System

Version 1.0 MVP

Xiaowei Wu

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms and Abbreviations
 - 1.4 References
2. System Overview
3. System Components
 - 3.1 Decomposition Description
 - 3.2 Dependency Description
 - 3.3 Interface Description
 - 3.3.1 Ingestion to Seed Provider Interface
 - 3.3.2 Seed Provider to Static Analyzer Interface
 - 3.3.3 Runtime Collector to Fusion Engine Interface
 - 3.3.4 Fusion Engine to Export Layer Interface
4. Detailed Design
 - 4.1 Module Detailed Design
 - 4.1.1 Ingest Project and SCA Result
 - 4.1.2 Load Vulnerable Method Seed
 - 4.1.3 Analyse Static Reachability
 - 4.1.4 Process Runtime Evidence
 - 4.1.5 Fuse Evidence and Generate Risk Report
 - 4.2 Data Detailed Design
 - 4.3 MVP Implementation Plan and Time Estimate

1. Introduction

1.1 Purpose

The Software Design Document describes the architecture and system design for the Evidence-driven Method-level Vulnerability Risk Assessment System. The system is designed to improve traditional component-level software composition analysis by adding method-level static reachability analysis, runtime evidence collection, evidence fusion, risk scoring, and structured report generation.

The purpose of this document is to define the system architecture, major components, data flow, module responsibilities, and implementation design for the MVP version of the system. This document is intended for the project supervisor, software engineers, security researchers, and anyone else who will be involved in reviewing, implementing, testing, or extending the system.

The MVP version focuses on proving that a component-level vulnerability alert can be transformed into a more context-aware risk decision. Instead of only reporting that a vulnerable dependency exists in a project, the system aims to determine whether a known vulnerable method is reachable from the application code and whether the resulting evidence is sufficient to generate a structured risk report.

1.2 Scope

This document describes the implementation design of the MVP version of the Evidence-driven Method-level Vulnerability Risk Assessment System. The system will focus on Java/Maven projects in the first stage and will use manually or semi-automatically confirmed vulnerable method seeds as input. The system will then perform static reachability analysis, combine the result with vulnerability and dependency evidence, and generate an evidence-backed risk decision.

The MVP system will consist of six major components:

1. Ingestion and Normalization Module

This module imports project dependency information, SBOM files, lockfiles, and SCA scan results, and normalizes them into a unified internal format. It converts heterogeneous scanner outputs and dependency metadata into standard project, component, dependency, and vulnerability records for later seed matching, reachability analysis, and evidence fusion.

2. Vulnerable Method Seed Provider

This module manages vulnerable method seed files. A seed maps a vulnerability identifier, such as a CVE or GHSA ID, to one or more vulnerable Java methods using the fully qualified class name, method name, and JVM method descriptor. In the MVP, these seeds are manually or semi-automatically confirmed and serve as the target methods for static reachability analysis.

3. **Static Reachability Analyzer**

This module analyzes the target Java/Maven project and determines whether a vulnerable method seed is statically reachable from application entry points. The MVP will implement this module as a wrapper around WALA, Soot, or SootUp, with the design informed by Shen et al.'s method-level reachability approach.

4. **Runtime Evidence Collector**

This module collects and imports runtime execution evidence, such as OpenTelemetry traces, to support validation of whether a vulnerable method or related execution path is observed during execution. In the MVP, the module focuses on trace data generated from controlled demo test runs, rather than production-level monitoring, and provides runtime evidence to the Evidence Fusion and Risk Scoring Engine.

5. **Evidence Fusion and Risk Scoring Engine**

This module combines component-level vulnerability evidence, vulnerable method seed evidence, static reachability evidence, and runtime evidence to produce a risk decision, confidence score, and explanation. It applies predefined decision rules and scoring weights to classify the vulnerability impact status, such as candidate, likely affected, affected, not affected candidate, or under investigation.

6. **Export and Report Generator**

This module generates structured outputs, including JSON risk reports and VEX-like vulnerability impact statements. The JSON risk report contains detailed evidence, risk scores, confidence scores, and explanations, while the VEX-like statement summarizes the vulnerability impact status for vulnerability management and documentation. Both outputs convert the final decision and supporting evidence into machine-readable formats for review, documentation, and further analysis.

The MVP version will not attempt to automatically locate vulnerable methods from arbitrary CVE descriptions. It will also not attempt to support all programming languages, all package ecosystems, production-scale runtime monitoring, or a full graphical dashboard. These features are considered future extensions. The initial system boundary is intentionally limited to a seed-based, semi-automated, evidence-driven vulnerability risk assessment pipeline, so that the MVP can focus on validating the core workflow from vulnerability evidence to reachability analysis, runtime evidence, risk scoring, and structured reporting.

1.3 Definitions, Acronyms and Abbreviations

Acronym	Meaning
SDD	Software Design Document
MVP	Minimum Viable Product

SCA	Software Composition Analysis
SBOM	Software Bill of Materials
CVE	Common Vulnerabilities and Exposures
GHSA	GitHub Security Advisory
VEX	Vulnerability Exploitability eXchange
WALA	T. J. Watson Libraries for Analysis
Soot	Java Optimization Framework / Static Analysis Framework
PURL	Package URL
FQCN	Fully Qualified Class Name
JVM	Java Virtual Machine
JSON	JavaScript Object Notation
CI	Continuous Integration
RTM	Requirements Traceability Matrix

Term	Meaning
Vulnerable Method Seed	A manually or semi-automatically confirmed mapping from a vulnerability identifier to one or more vulnerable Java methods.
Static Reachability	The analysis of whether a target method can be reached from application entry points through the call graph.
Runtime Evidence	Execution evidence collected from test runs or traces showing whether a method or path was observed during execution.
Evidence Fusion	The process of combining SCA evidence, seed evidence, static reachability evidence, and runtime evidence to generate a risk decision.

1.4 References

- [1] Y. Shen, X. Gao, H. Sun, and Y. Guo, “Understanding vulnerabilities in software supply chains,” *Empirical Software Engineering*, vol. 30, no. 1, p. 20, 2025.
- [2] H. Plate, S. E. Ponta, and A. Sabetta, “Impact assessment for vulnerabilities in open-source software libraries,” in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2015, pp. 411–420.
- [3] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, “Vuln4Real: A methodology for counting actually vulnerable dependencies,” *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592–1609, 2020.
- [4] R. Buan, Z. Lin, J. Liu, C. Zhang, and K. J. Z. Liang, “Propagation-based vulnerability impact assessment for software supply chains,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 65–77.

- [5] L. Zhou, M. Dacier, and C. Konstantinou, "A reality check on SBOM-based vulnerability management: An empirical study and a path forward," arXiv preprint arXiv:2511.20313, 2025.
- [6] M. Mirakhorli, D. Garcia, S. Dillon, K. Laporte, M. Morrison, H. Lu, V. Koscinski, and C. Enoch, "A landscape study of open source and proprietary tools for software bill of materials (SBOM)," arXiv preprint arXiv:2402.11151, 2024.
- [7] S. Elder, M. R. Rahman, G. Fringer, K. Kapoor, and L. Williams, "A survey on software vulnerability exploitability assessment," ACM Computing Surveys, vol. 56, no. 8, pp. 1–41, 2024.
- [8] P. Mell, K. Scarfone, and S. Romanosky, "Common vulnerability scoring system," IEEE Security & Privacy, vol. 4, no. 6, pp. 85–89, 2007.
- [9] The White House, "Executive Order 14028: Improving the Nation's Cybersecurity," Federal Register, May 12, 2021.
- [10] OpenTelemetry Documentation.
- [11] WALA Documentation.
- [12] Soot / SootUp Documentation.
- [13] CycloneDX SBOM and VEX Documentation.
- [14] OpenVEX Specification.

2. System Overview

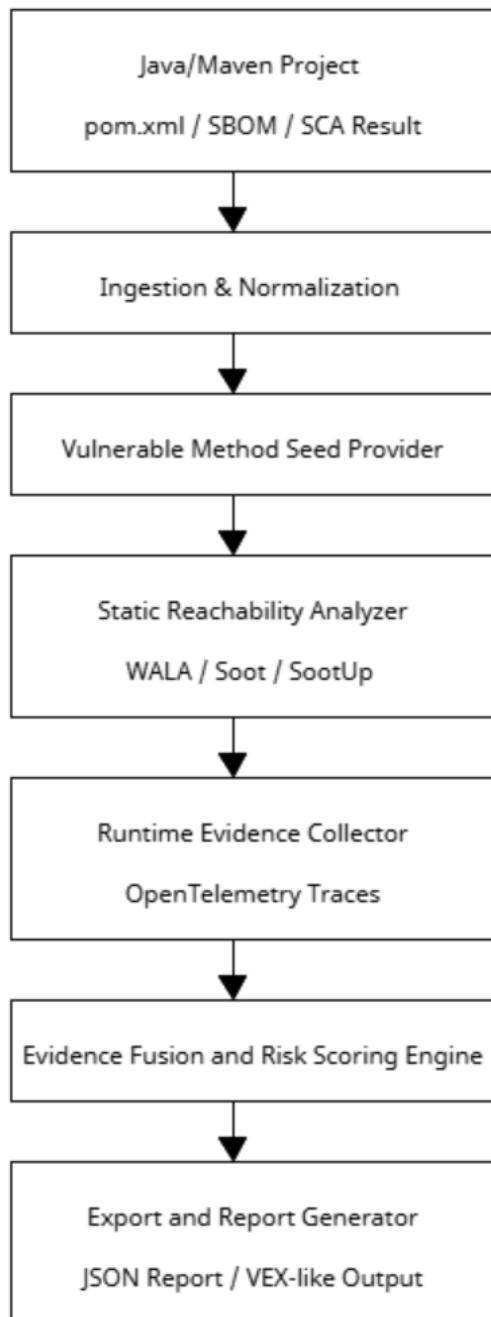


Figure 1

Figure 1 represents the high-level architecture of the Evidence-driven Method-level Vulnerability Risk Assessment System. The system is designed as a vulnerability analysis pipeline that transforms component-level vulnerability alerts into evidence-backed risk decisions.

The system receives input from Java/Maven projects, SBOM files, dependency metadata, lockfiles, and SCA scan results. The Ingestion and Normalization module

converts these heterogeneous inputs into a unified internal format, including project, component, dependency, and vulnerability records.

After the dependency and vulnerability information is normalized, the Vulnerable Method Seed Provider maps a vulnerability identifier, such as a CVE or GHSA ID, to one or more vulnerable Java methods. These vulnerable method seeds are then used by the Static Reachability Analyzer to determine whether the vulnerable method is reachable from the application entry points. The MVP implementation may use an existing Java static analysis framework, such as WALA, Soot, or SootUp, through a project-specific analyzer wrapper.

The Runtime Evidence Collector processes OpenTelemetry trace data generated from controlled test executions. This runtime evidence is used to validate whether a vulnerable method or related execution path is observed during actual execution. The Evidence Fusion and Risk Scoring Engine then combines component-level vulnerability evidence, vulnerable method seed evidence, static reachability evidence, and runtime evidence to produce a risk decision, confidence score, and explanation.

Finally, the Export and Report Generator produces machine-readable outputs, including a detailed JSON risk report and a simplified VEX-like vulnerability impact statement. These outputs support review, documentation, and further vulnerability management.

3. System Components

3.1 Decomposition Description

The Evidence-driven Method-level Vulnerability Risk Assessment System is decomposed into six main components: Ingestion and Normalization, Vulnerable Method Seed Provider, Static Reachability Analyzer, Runtime Evidence Collector, Evidence Fusion and Risk Scoring Engine, and Export and Report Generator.

The **Ingestion and Normalization Module** receives project dependency information, SBOM files, lockfiles, and SCA scan results. It converts heterogeneous dependency and vulnerability data into standardized project, component, dependency, and vulnerability records.

The **Vulnerable Method Seed Provider** manages seed files that map vulnerability identifiers, such as CVE or GHSA IDs, to vulnerable Java methods. These seeds provide the target method signatures used by the Static Reachability Analyzer.

The **Static Reachability Analyzer** analyzes the target Java/Maven project and determines whether a vulnerable method seed is reachable from application entry points. It is implemented as a wrapper around an existing Java static analysis framework, such as WALA, Soot, or SootUp.

The **Runtime Evidence Collector** processes OpenTelemetry trace data generated from controlled test executions. It identifies whether a vulnerable method or related execution path is observed during execution and provides runtime evidence to the fusion engine.

The **Evidence Fusion and Risk Scoring Engine** is the central decision-making component. It combines component-level vulnerability evidence, vulnerable method seed evidence, static reachability evidence, and runtime evidence to produce a risk decision, confidence score, and explanation.

The **Export and Report Generator** converts the final decision and supporting evidence into machine-readable outputs, including JSON risk reports and VEX-like vulnerability impact statements.

For the MVP version, the system will prioritize a minimal end-to-end pipeline: ingesting vulnerability and dependency evidence, loading vulnerable method seeds, performing static reachability analysis, processing controlled runtime trace evidence, and generating structured risk reports. A graphical architecture diagram may be added in the final version of the document.

3.2 Dependency Description

Component	Depends On	Produces
-----------	------------	----------

Ingestion and Normalization	Project metadata, SBOM, lockfile, SCA result	Normalized project, component, dependency, and vulnerability records
Vulnerable Method Seed Provider	Normalized CVE/GHSA and component records, seed files	Vulnerable method signatures
Static Reachability Analyzer	Java/Maven project, classpath, vulnerable method signatures	Static reachability evidence
Runtime Evidence Collector	OpenTelemetry traces from controlled test executions	Runtime observation evidence
Evidence Fusion and Risk Scoring Engine	Component evidence, seed evidence, static evidence, runtime evidence	Risk decision, confidence score, explanation
Export and Report Generator	Final risk decision and supporting evidence	JSON risk report, VEX-like statement

The system components are organized as an evidence-processing pipeline. Each component depends on the structured output produced by the previous analysis stage, and the Evidence Fusion and Risk Scoring Engine depends on multiple evidence sources to generate the final decision.

The **Ingestion and Normalization Module** is the first component in the pipeline. It depends on external project inputs, including Java/Maven project metadata, SBOM files, lockfiles, and SCA scan results. Its normalized output is required by the Vulnerable Method Seed Provider, the Static Reachability Analyzer, and the Evidence Fusion and Risk Scoring Engine.

The **Vulnerable Method Seed Provider** depends on normalized vulnerability identifiers and component information. It matches vulnerability records, such as CVE or GHSA IDs, with vulnerable method seed files. The output of this module is a set of target vulnerable method signatures that will be used by the Static Reachability Analyzer.

The **Static Reachability Analyzer** depends on the target Java/Maven project, the project classpath, and the vulnerable method signatures provided by the Seed Provider. It produces static reachability evidence, including the reachability status, analysis engine, and possible call paths.

The **Runtime Evidence Collector** depends on trace data generated from controlled test executions. It processes OpenTelemetry trace data and produces runtime evidence indicating whether a vulnerable method or related execution path was observed during execution.

The **Evidence Fusion and Risk Scoring Engine** depends on four types of evidence: component-level vulnerability evidence from the Ingestion module, vulnerable method seed evidence from the Seed Provider, static reachability evidence from the Static Reachability Analyzer, and runtime evidence from the

Runtime Evidence Collector. It combines these inputs to produce a risk decision, confidence score, and explanation.

The **Export and Report Generator** depends on the final output of the Evidence Fusion and Risk Scoring Engine. It converts the decision, scores, and supporting evidence into machine-readable reports, including JSON risk reports and VEX-like vulnerability impact statements.

3.3 Interface Description

This section describes the major interfaces between system components. In the MVP version, the interfaces are mainly implemented through structured data objects, such as JSON files, YAML seed files, and internal Python or Java data models. These interfaces define how evidence flows from one module to another.

3.3.1 Ingestion to Seed Provider Interface

The Ingestion and Normalization Module provides normalized vulnerability and component records to the Vulnerable Method Seed Provider. The Seed Provider uses the vulnerability identifier and component information to locate matching vulnerable method seeds.

Input: normalized component record, vulnerability ID, package metadata

Output: matched vulnerable method seed candidates

Example fields:

component_purl
group_id
artifact_id
version
vulnerability_id
source

3.3.2 Seed Provider to Static Analyzer Interface

The Vulnerable Method Seed Provider sends target vulnerable method signatures to the Static Reachability Analyzer. These method signatures are used as analysis targets in the call graph.

Input: vulnerable method seed

Output: analyzer-compatible target method signature

Example fields:

fqn
method_name
jvm_descriptor
cve_id
seed_confidence

3.3.3 Runtime Collector to Fusion Engine Interface

The Runtime Evidence Collector provides runtime observation evidence to the Evidence Fusion and Risk Scoring Engine. This evidence indicates whether a vulnerable method or related execution path was observed during controlled test execution.

Input: OpenTelemetry trace data

Output: runtime evidence record

Example fields:

trace_id
span_name
observed_method
environment
runtime_status

3.3.4 Fusion Engine to Export Layer Interface

The Evidence Fusion and Risk Scoring Engine sends the final risk decision, confidence score, and supporting evidence to the Export and Report Generator.

Input: evidence fusion result

Output: JSON risk report and VEX-like vulnerability impact statement

Example fields:

decision_status
risk_score
confidence_score
evidence_level
explanation
recommended_action

4. Detailed Design

4.1 Module Detailed Design

This section describes the detailed design of the main MVP workflows. Each workflow corresponds to one major analysis step in the evidence-driven vulnerability risk assessment pipeline.

4.1.1 Ingest Project and SCA Result

Purpose:

This workflow imports project dependency information and component-level vulnerability findings into the system.

Input:

Java/Maven project metadata, SBOM files, lockfiles, and SCA scan results.

Process:

The system reads the project dependency information and scanner outputs, extracts component and vulnerability records, and normalizes them into a unified internal format.

Output:

Normalized project, component, dependency, and vulnerability records.

Pseudocode:

```
Load project metadata
Load SBOM or dependency information
Load SCA scan result
Extract component records
Extract vulnerability records
Normalize package identifiers into PURL format
Store normalized records for later analysis
```

4.1.2 Load Vulnerable Method Seed

Purpose:

This workflow loads vulnerable method seed files and matches them with normalized vulnerability records.

Input:

Vulnerability ID, component information, and seed files.

Process:

The system identifies seed files that match the vulnerability identifier and affected component. Each seed provides the fully qualified class name, method name, and JVM method descriptor of a vulnerable Java method.

Output:

Matched vulnerable method signatures.

Pseudocode:

```
Load normalized vulnerability records
Load vulnerable method seed files
For each vulnerability:
    Find matching seed by CVE or GHSA ID
    Validate component and package information
    Extract vulnerable method signature
Return target method signatures
```

4.1.3 Analyze Static Reachability**Purpose:**

This workflow determines whether a vulnerable method seed is statically reachable from application entry points.

Input:

Target Java/Maven project, project classpath, application entry points, and vulnerable method signatures.

Process:

The Static Reachability Analyzer invokes an existing Java static analysis framework, such as WALA, Soot, or SootUp. It constructs or queries a call graph and searches for paths from selected application entry points to the target vulnerable methods.

Output:

Static reachability evidence, including reachable, unreachable, or unknown status, possible call paths, analysis engine, and confidence information.

Pseudocode:

```
Load target project classpath
Load vulnerable method signatures
Identify application entry points
Invoke static analysis framework
Build or query call graph
For each target vulnerable method:
    Search for path from entry point to target method
    If path exists:
        Mark status as reachable
        Record call path
    Else if analysis is incomplete:
        Mark status as unknown
    Else:
```

Mark status as unreachable
Return static reachability evidence

4.1.4 Process Runtime Evidence

Purpose:

This workflow processes runtime trace data generated from controlled test executions.

Input:

OpenTelemetry trace data, trace identifiers, span names, and optional method-related attributes.

Process:

The Runtime Evidence Collector reads trace data from controlled demo test runs and checks whether a vulnerable method or related execution path was observed during execution.

Output:

Runtime evidence record, including observed or not observed status, trace ID, span information, and execution environment.

Pseudocode:

```
Load runtime trace data
Load vulnerable method signatures
For each trace:
    Inspect span names and method-related attributes
    If vulnerable method or related path appears:
        Mark runtime status as observed
        Record trace ID and span information
    Else:
        Mark runtime status as not observed
Return runtime evidence
```

4.1.5 Fuse Evidence and Generate Risk Report

Purpose:

This workflow combines all available evidence and generates the final risk decision and report.

Input:

Component-level vulnerability evidence, vulnerable method seed evidence, static reachability evidence, and runtime evidence.

Process:

The Evidence Fusion and Risk Scoring Engine applies predefined decision rules and scoring weights. It classifies the vulnerability impact status and generates a

confidence score and explanation. The Export and Report Generator then converts the result into a JSON risk report and a VEX-like vulnerability impact statement.

Output:

Risk decision, confidence score, explanation, JSON risk report, and VEX-like statement.

Pseudocode:

Load component-level vulnerability evidence

Load vulnerable method seed evidence

Load static reachability evidence

Load runtime evidence

If component exists and seed exists and static status is reachable and runtime status is observed:

 Decision = affected

Else if component exists and seed exists and static status is reachable:

 Decision = likely affected

Else if component exists and seed exists and static status is unreachable:

 Decision = not affected candidate

Else:

 Decision = under investigation

Calculate risk score

Calculate confidence score

Generate explanation

Export JSON risk report

Export VEX-like statement

4.2 Data Detailed Design

Data Object	Description	Key Fields
Project	Target software project being analysed	project_id, name, version, build_system
Component	Software dependency or package	purl, group_id, artifact_id, version, scope
Vulnerability	Vulnerability record from SCA or advisory source	vulnerability_id, source, severity, affected_range
VulnerableMethodSeed	Mapping from vulnerability to vulnerable Java method	cve_id, fqcn, method_name, jvm_descriptor, confidence
StaticReachabilityEvidence	Result of static reachability analysis	status, entrypoint, target_method, call_path, engine
RuntimeEvidence	Runtime observation from controlled test traces	trace_id, span_name, observed_method, runtime_status, environment
RiskDecision	Final vulnerability impact decision	decision_status, risk_score, confidence_score, explanation

Report	Machine-readable output document	report_id, format, generated_at, evidence_refs
--------	----------------------------------	--

4.3 MVP Implementation Plan and Time Estimate

This section provides an estimated implementation schedule for the MVP version of the system. The MVP focuses on validating the core end-to-end workflow: ingesting project and vulnerability evidence, loading vulnerable method seeds, performing static reachability analysis, processing controlled runtime trace evidence, fusing evidence, and generating structured reports.

Phase	Task	Main Component	Estimated Time
Phase 1	Project setup and repository structure	Overall System	1–2 days
Phase 2	Ingest project dependency and SCA result	Ingestion and Normalization Module	3–5 days
Phase 3	Define and load vulnerable method seed files	Vulnerable Method Seed Provider	2–3 days
Phase 4	Implement static reachability analyzer wrapper	Static Reachability Analyzer	1–2 weeks
Phase 5	Generate runtime trace data from controlled test runs	Runtime Evidence Collector	4–7 days
Phase 6	Implement evidence fusion rules and risk scoring	Evidence Fusion and Risk Scoring Engine	4–6 days
Phase 7	Generate JSON risk report and VEX-like statement	Export and Report Generator	3–5 days
Phase 8	End-to-end MVP testing and refinement	Whole Pipeline	1 week

Total estimated time: approximately **5–7 weeks** for a functional MVP prototype.

The most technically demanding part of the MVP is the Static Reachability Analyzer, because it requires integrating an existing Java static analysis framework such as WALA, Soot, or SootUp and adapting its output to the system’s evidence model. The Runtime Evidence Collector is expected to be implemented in a controlled test environment rather than as production-level monitoring. Therefore, the MVP will focus on processing traces generated from predefined demo test executions.

For the MVP, the implementation priority is to complete a minimal end-to-end pipeline before optimizing individual modules. Advanced features, such as automatic CVE-to-method localization, multi-language support, production-scale monitoring, a full dashboard, and large-scale empirical evaluation, are considered future work.